

SYSTEM ANALYSIS DOCUMENTATION (SAD)

UMHackathon 2026

umhackathon@um.edu.my

1. Introduction:

MindaFinancial introduces a strategic AI-driven financial intelligence solution for Malaysian F&B SMEs. It addresses the "Insight Gap" where entrepreneurs possess raw data (receipts/POS logs) but lack the analytical capacity to optimize budgets, predict stock "rushes," and reduce waste.

1.1. Purpose:

This document highlights the technical scope and design decisions behind MindaFinancial. Key elements covered:

So, the key element of system that has been covered are as followed:

1. Architecture:

- a. A cloud-native microservices architecture leveraging a GLM-as-a-Service layer for financial reasoning.

2. Data Flows:

- a. Details how unstructured receipt data flows from ingestion to GLM-structured JSON, into PostgreSQL, and finally to the user dashboard.

3. Model Process:

- a. End-to-end workflow for automated inventory forecasting and budget optimization.

4. Role of Reference:

- a. A high-level blueprint for developers and testers to ensure system integrity during the 10-day sprint.

1.2. Background:

Small F&B businesses in Malaysia operate on razor-thin margins. While digital payment adoption (DuitNow/GrabPay) is high, financial planning remains manual and reactive. MindaFinancial bridges this by turning "boring" records into an active "AI-CFO."

1. Previous Version:

- a. Traditional SME tools are static accounting ledgers. They require manual entry and offer no forward-looking "Decision Narrative."

2. Changes in Major Architectural Components

3. New capabilities that are only introduced in this

- a. MindaFinancial introduces **Automated Data Extraction** (unstructured to structured) and **Agentic Forecasting** (predicting demand based on external events like Hari Raya or Monsoon weather).

1.3. Target Stakeholder

<i>Stakeholders</i>	<i>Roles</i>	<i>Expectations</i>
<i>F&B Owner</i>	<i>Approves budgets and monitors health.</i>	<i>High-level "Decision Narratives" and RM-based savings.</i>
<i>Ops Manager</i>	<i>Manages stock and vendor orders.</i>	<i>Accurate "Rush Predictions" and traffic-light stock alerts.</i>
<i>Development Team</i>	<i>Build/Maintain Python & Node services.</i>	<i>Clean API contracts between the GLM and Backend.</i>
<i>QA Team</i>	<i>Validate AI accuracy & system uptime.</i>	<i>Minimal "hallucinations" in financial calculations.</i>

2. System Architecture & Design

2.1. High Level Architecture

2.1.1. Overview:

Type	Details
System	Web-based Dashboard (Responsive React.js).
Architecture	Cloud-Server with Microservices (Node.js for logic, Python for AI).

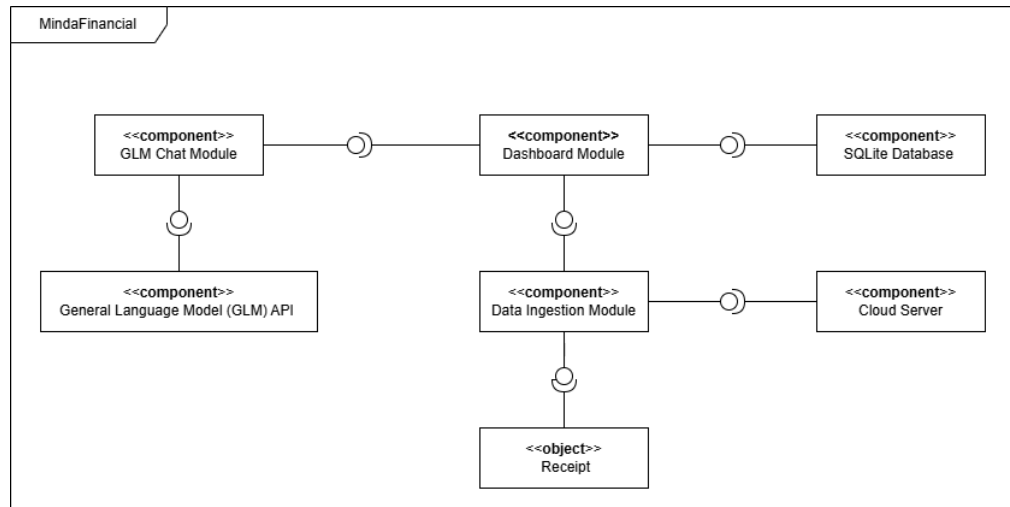
MindaFinancial is structured as a cloud-native microservices system. The primary client is a Responsive Web-based Dashboard used by F&B owners and managers. This client communicates via a centralized Backend API Gateway (Node.js), which manages user sessions and database persistence. To handle complex financial reasoning, the system employs a specialized AI Reasoning Microservice (Python) that acts as the bridge to the GLM Service Layer. Both services are deployed on Oracle Cloud Infrastructure (OCI), utilizing PostgreSQL for structured financial records and AWS S3 for secure storage of raw receipt imagery.

2.1.2 LLM as Service Layer

The architecture of your product needs to show how GLM is integrated as a service layer- not limited to a generic “AI Module”. The following Dependency diagram includes:

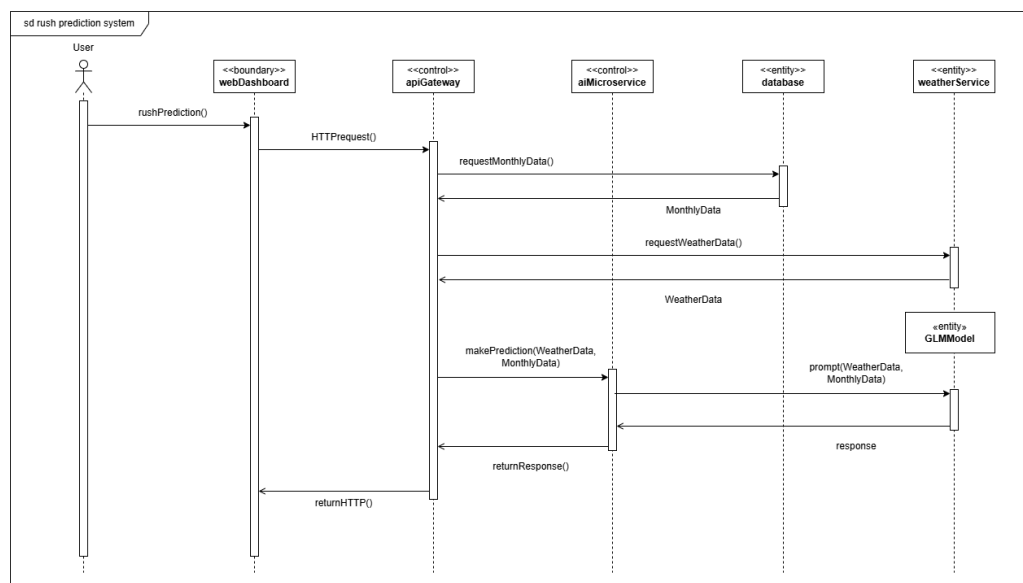
2.1.3 Dependency Diagram

The dependencies of the MindaFinancial system is represented below using a UML Component Diagram:



2.2.2. Sequence Diagram

For the Sequence Diagram, it is representing the use case of making a rush prediction:



2.2. Technological Stack

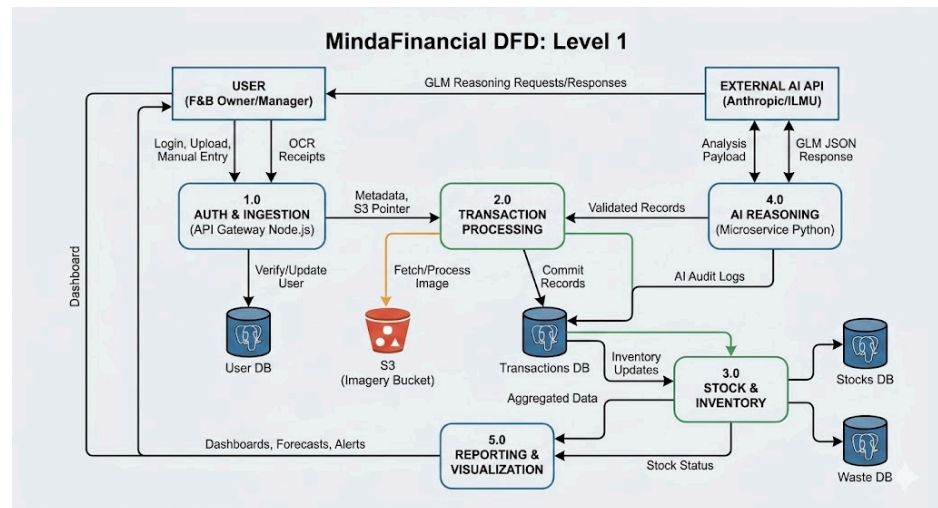
- 2.2.1. Frontend: React.js (Professional, high-contrast dashboard using Recharts).
- 2.2.2. Backend: Node.js (Express) & Python (FastAPI for AI microservice).
- 2.2.3. Database: PostgreSQL (via Oracle Cloud/Supabase) & AWS S3 for raw receipt images.
- 2.2.4. Cloud/Deployment: Oracle Cloud (Always Free Tier) for Compute/RAM; AWS for Storage.

2.3. Key Data Flows

2.3.1. Here, do show the how data are moving through the system and how its structure and stored

2.3.1.1. Data Flow Diagram (DFD)

a.



2.3.2. Normalized Database Schema

a. Schema:

User (id, email, name, role, createdAt, updatedAt)

Vendor (id, name, email, phone, category, status, userId, createdAt, updatedAt)

Transaction (id, amount, type, category, description, date, status, vendorId, userId, createdAt, updatedAt)

Stock (id, name, quantity, unit, category, reorderLevel, lastRestocked, createdAt, updatedAt)

Waste (id, item, quantity, unit, reason, cost, date, createdAt, updatedAt)

RushOrder (id, customerName, items, totalAmount, status, priority, notes, orderDate, completedAt, createdAt, updatedAt)

b. Relationship constraints

1. User.id -> Vendor.userId
2. User.id -> Transaction.userId
3. Vendor.id -> Transaction.vendorId

3. Functional Requirements & Scope

This section highlights the core boundaries of the projects. That enables them to focus on a high-impact MVP to showcase the technical feasibility of the project.

3.1. Minimum Viable Product:

3.1.1. The features that must be implemented in this phase of development are presented in the table below:

#	Features	Description
1	Smart Data Ingestion	Upload a receipt/invoice; GLM extracts Date, Vendor, Amount, and Category.
2	Traffic Light Stock	Real-time alerts (Red/Yellow/Green) for core ingredients (e.g., Chicken, Rice).
3	Rush Prediction	Forecasts customer volume for the next 24 hours based on historical data + weather.
4	AI-CFO Dashboard	A narrative-driven summary: "You can save RM200 this week by reducing Oil orders."

3.2. Non-Functional Requirements (NFRs)

Here, highlights the system qualities that are not features but are crucial for reliability and improve the system. [e.g. table in below]

Quality (e.g.)	Requirements (e.g.)	Implementation (e.g)
Accessibility	The dashboard must be usable across multiple devices (Mobile/Tablet/PC).	Responsive UI Design using Tailwind CSS and a "Mobile-First" approach for busy kitchen staff and on-the-go managers.
Reliability	The system must maintain 99% uptime during the demo, with no orphaned records.	Database operations use ACID-compliant transactions; if an AI extraction fails, the database automatically rolls back the partial entry.
Maintainability	The system must allow for rapid updates to the AI model or business logic without downtime.	The AI Reasoning service is separate from the Core Business Logic, allowing for independent CI/CD pipelines.

Token Latency	AI reasoning and data extraction must return a response within 3s-5s at P95 .	Use Async API calls and FastAPI for non-blocking I/O. Implement frontend skeleton loaders to maintain perceived performance.
Cost Efficiency	Average token cost per user session must not exceed RM 0.50 to remain viable for SMEs.	Implement Prompt Engineering with fixed-length outputs and Redis Caching for recurring analytics queries.

3.3. Out of Scope / Future Enhancements

- a. **Predictive "What-If" Scenario Simulation** : Future iterations will feature a Financial Sandbox where owners can simulate business decisions. For example: "If I increase my staff count by 2 and open on Public Holidays, what is the projected impact on my 30-day cash flow?" This moves the tool from reactive reporting to proactive strategy simulation using advanced Agentic AI.
- b. **Multi-Outlet Centralized Command** : The current version is optimized for a single-location SME. The future enhancement involves a Franchise Analytics Layer, allowing a headquarters to monitor inventory waste and financial health across 50+ locations simultaneously, identifying which outlets are underperforming using cross-locational AI benchmarking.
- c. **Automated Procurement & B2B Settlement** : While the current MVP allows for "One-Click Approval" of stock orders, the "Grand Vision" includes a direct API Handshake with suppliers. This would allow the system to automatically negotiate prices, place orders, and facilitate digital payments (DuitNow/FPX) without human intervention when stock hits a critical "Red" level.

4. Monitor, Evaluation, Assumptions & Dependencies

4.1. Technical Evaluation:

- 4.1.1. **Grayscale Rollout & A/B Testing**: For greyscale rollout, the chosen strategy for testing is to use a OCI API gateway or a Service Mesh to route only 5% of traffic to the new Python AI Microservice version. The health of the system is evaluated by monitoring the AI reasoning latency and PostgreSQL "Commit Success Rate". The benchmark for rollout of newer versions requires a spike in latency of less than 500ms.

For A/B testing, an experiment is conducted by testing two different GLM prompts or models to differentiate which one parses receipt data with higher accuracy. Segmented routing is implemented for higher tiered users to ensure higher accuracy could be achieved compared to the baseline users.

- 4.1.2. **Emergency Rollback (ER) & Golden Release:** For the emergency rollback trigger, it is automatically alerted via OCI monitoring. For example, the “Financial Transactions” threshold exceeds 0.1% over a 2 minute period, the ER is triggered. In this scenario, the Load Balancer will immediately sever traffic and redirect traffic to a stable environment. For the Golden Release, the system must pass the threshold of a simulated 7-day stress test. The strategy to implement golden release is having the OCI to maintain a “Golden Image” in the OCI Container Registry. When ER is triggered, the pipeline will not “revert code” but instead redeploy the specific Golden Image known to be compatible with the system structure.

4.1.3. Priority Matrix:

- 4.1.3.1. For MindaFinancial, the priority matrix must bridge the gap between financial integrity and inherent unpredictability of LLM-based reasoning.

Priority Level	Severity	Trigger Conditions (Monitoring Alerts)	Automated/Manual Actions
P0: Critical	Systemic Failure	1. Core database unreachable 2. Unauthorized AWS S3 access detected	1. Trigger the Emergency Rollback 2. Traffic Kill Switch
P1: High	Operational Disruption	1. AI Reasoning latency > 15s (P95) 2. GLM APO returns 401/403 errors 3. “No Importer Manifest: build errors in CI/CD	1. Circuit Breaker Open 2. Greyscale Rollback
P2: Medium	Degraded Experience	1. Receipt OCR accuracy drops below 90% through A/B	1. Load Balancer Re-route 2. Cache Purge

		testing 2. OCI-to-AWS cross-cloud latency > 500ms 3. UI Dashboard rendering errors > 2%	
P3: Low	Maintenance/Optimization	1. Non-critical UI styling bugs 2. S3 storage optimization alerts 3. Minor logs indicating missing metadata	1. Log and Observe 2. Patch Release

4.2. Monitoring:

4.2.1. The monitoring for MindaFinancial uses an industry standard for multi-cloud systems which is Predictive Observability. The monitoring plan is defined by four key components that also defines the damage-detection trigger logic:

- The Multi-Cloud Pulse
 1. The infrastructure such as OCI Monitoring and CloudWatch, tracks the CPU, memory and OCI-to-AWS network latency.
 2. The applications such OpenTelemetry and Jaeger can trace a single receipt entry from the Gateway to the AI service. If the trace cannot be tracked or returns a 404/500 status code, we can know which microservice
 3. The AI quality by observing the LLM specifically monitors the GLM API responses for faithfulness and hallucination rates.
- The Damage Detection Trigger Logic
 1. For the data integrity layer, the triggers are orphaned S3 objects that can be detected using OCI scheduled tasks.
 2. For the AI reliability layer, the trigger is an empty response from the GLM that can be detected by Python health checks.
 3. For the connectivity layer, the trigger is a cross-cloud latency spike that can be detected by OCI health checks
 4. Lastly, for the systemic health layer, the trigger are cascading timeouts that can be detected using the API gateway circuit breaker

- Automated Response Plan
 1. Phase 1: Isolation which takes 0-10 seconds that can be triggered by AI latency or a connection refusal from the DB. The recommended action for this is having the OCI Load Balancer stop sending traffic to the damaged code and shift the traffic for the Golden Version.
 2. Phase 2: Preservation which takes 10-60 seconds that can be triggered by missing data or importer manifest errors in logs. The actions for this phase is making the system enter write-only mode (users can upload receipts but AI reasoning is stopped to prevent SQL corruption).
 3. Phase 3: Restoration which takes 1-5 minutes that can be triggered by the metrics of the system health is within the green zone via the Golden Release. The recommended action for this phase is for the CI/CD pipeline to trigger a replay of pending S3 images through the AI service to catch up on backlogs and ensure zero data loss.
- Visualizing the Health Dashboard
 1. The health of the dashboard is visualized using a traffic light dashboard
 2. The states of the traffic light shows the different levels of eros that can happen in the system (e.g: Yellow is the warning signal from a slowed response from the GLM API)

4.3. Assumptions: The key operational & environmental conditions that is assumed to be valid while development and deployment of MindaFinancial include:

1. The network latency between the OCI and AWS stays below 150ms
2. OSI-S3 Compatibility
3. Persistence Availability for ManagedPostgreSQL
4. Stateless AI Service
5. Token window limits will not exceed 128k tokens context window
6. Non-deterministic outputs
7. The schema is synchronized for the [Node.js](#) and Python implementations
8. The “Golden Image” remains integral throughout deployment
9. The staging environment is perfectly mirroring the OCI-AWS cloud implementations

4.4. External Dependencies:

Tools	Purpose	Risks
Z Ai GLM API	Serves as the primary large language model (LLM) engine, responsible for processing user prompts, performing multi-step reasoning, and generating structured outputs.	If the API experiences rate limits or downtime, core system functionality may be disrupted. Mitigation strategies should include

		implementing fallback mechanisms such as a graceful degradation mode to maintain basic functionality for users.
Prisma ORM	Acts as the main database ORM to manage models (User, Vendor, Transaction, etc.) and handle queries between application and database.	Medium – breaking schema changes or migration errors can corrupt data or cause downtime
SQLite	Lightweight database used to store all system data (transactions, stock, waste, orders)	Medium – not scalable for large production systems; risk of performance bottlenecks
Node.js	Backend runtime to execute the server and business logic	Medium – crashes or memory leaks can affect entire system availability
GitHub	Version control and collaboration for development	Low – outages may slow development but not runtime system
Vercel	Hosting and deployment of backend/frontend services	High – downtime will make the entire system inaccessible
dotenv	Manages sensitive environment variables like database URL	Medium – misconfiguration can break database connection
JSON	Used to store structured data (e.g. rush order items)	Low – malformed JSON may cause parsing errors

5. Project Management & Team Contributions

The duration of the Preliminary round is approximately 10 days considering the two-week sprint.

5.1. Project Timeline:

 MindaFinancial Project Plan

NO.	TASKS	PIC	PROGRESS	PLANNING		PREPARATION		EXECUTION			REFINEMENT & DELIVERY		
				DAY 1	DAY 2	DAY 3	DAY 4	DAY 5	DAY 6	DAY 7	DAY 8	DAY 9	DAY 10
1	Project Planning												
1.1	Discuss and finalize the idea of the project.	Everyo... ▾	Done ▾										
1.2	Elect project leader, secretary and treasurer	Everyo... ▾	Done ▾										
1.3	Create a centralized folder for project documents and materials	Aliq ▾	Done ▾										
1.4	Discuss the Product Requirement Document	Everyo... ▾	Done ▾										
1.5	Discuss the System Architecture Design and Technical Architecture Design	Everyo... ▾	Done ▾										
2	Project Preparation												
2.1	Draft technical framework and database design	Aliq ▾	Done ▾										
2.2	API Key Management	Eireen ▾	Done ▾										
2.3	Multimodal Ingestion Pipeline (Backend)	Nazhan ▾	Done ▾										
2.4	UI Scaffolding & State Management (Frontend)	Nia ▾	Done ▾										
2.5	Logic Integration (Analytics)	Fatimah ▾	Done ▾										
3	Project Execution												
3.1	Narrative Engine	Eireen ▾	Done ▾										
3.2	Deduplication Logic	Nazhan ▾	Done ▾										
3.3	Inventory & Ruah Prediction	Nia ▾	Done ▾										
3.4	Vendor & Waste Tracking	Fatimah ▾	Done ▾										
4	Refinement & Delivery												
4.1	QA & Feedback Testing	Aliq ▾	Done ▾										
4.2	Post-Mortem & Final Reporting	Everyo... ▾	Done ▾										

5.2. Team Members Role:



5.3. **Recommendations:** *If any recommendation is applicable in terms of scalability, performance and reliability [e.g. using Redis Cache for Menu data]*